MODULE 6 · LECTURE 27b

CS 6530 Applied Cryptography

# Post-Quantum Cryptography Foundations

*Module 6 –* Post-Quantum Cryptography

# From Polynomials to Modules — $(R_q^k)$

A module generalizes a vector space: its vectors can be added and multiplied by scalars from a ring (here, the polynomial ring $R_q$).

Just as an ordinary vector contains numbers, a module vector can contain polynomials.

**Recall:**

$$R_q = \mathbb{Z}_q[x] / (x^n + 1)$$

*Each element of $R_q$ is a polynomial of degree less than n.*

**New mathematical object:**

$$R_q^k = R_q \times R_q \times \cdots \times R_q \quad \text{(k copies)}$$

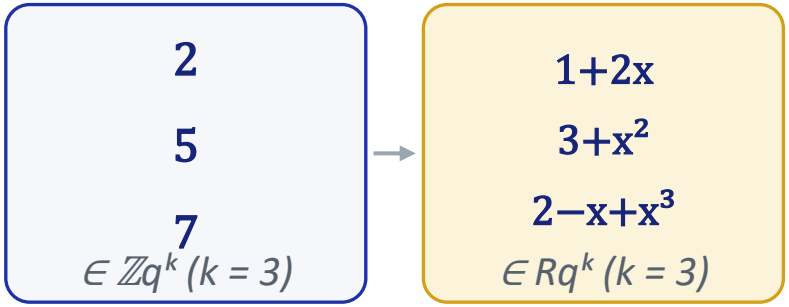
An element of $Rq^k$ is a column vector containing k polynomials:

$\boldsymbol{a} = (a_1(x), a_2(x), \ldots, a_k(x))^T, \quad a_i(x) \in R_q$

**Illustration — k = 3, n = 4:**

$$\boldsymbol{a} = (1+2x,\ 3+x^2,\ 2-x+x^3)^T \in R_q^3$$

**One number per entry → one polynomial per entry**




*The polynomial vector on the right has exactly the same shape as the ordinary vector on the left — only the entries change from numbers to polynomials.*

*Addition and subtraction are component-wise; each component is calculated using arithmetic in Rq.*

# Module Arithmetic — Addition and Inner Products

Module vectors support familiar vector operations, but each entry is a polynomial in Rq.

Compact example — $q = 7$, $n = 2$, $R_q = \mathbb{Z}_7[x] / (x^2 + 1) \implies x^2 = -1$

$\boldsymbol{a} = (1+x,\ 2-x)^T$, $\boldsymbol{b} = (2+x,\ 1+2x)^T$, both $\in R_q^2$

## Operation 1 — Vector addition

$\boldsymbol{a}+\boldsymbol{b} = ((1+x)+(2+x),\ (2-x)+(1+2x))^T$

$$\boldsymbol{a}+\boldsymbol{b} = (3+2x,\ 3+x)^T$$

*The result is another element of Rq².*

## Operation 2 — Inner product

$\boldsymbol{a}^T\boldsymbol{b} = (1+x)(2+x) + (2-x)(1+2x)$

$(1+x)(2+x) = 2+3x+x^2 = 1+3x$ (using $x^2=-1$)

$(2-x)(1+2x) = 2+3x-2x^2 = 4+3x$

$$\boldsymbol{a}^T\boldsymbol{b} = 5+6x \text{ in } R_q$$

$\boldsymbol{a}+\boldsymbol{b} \in R_q^2$ but $\boldsymbol{a}^T\boldsymbol{b} \in R_q$ — never confuse an inner product with component-wise multiplication.

# Symmetric Mod (Centered Modular Representation)

Integers that differ by a multiple of **q** are congruent modulo **q**. Symmetric mod chooses representatives close to zero, using negative as well as positive integers.

The same residue modulo q can be represented by a nonnegative integer or by a small signed integer.

**Ordinary representation (q = 17):**

$\mathbb{Z}_{17} = \{0, 1, \ldots, 16\}$

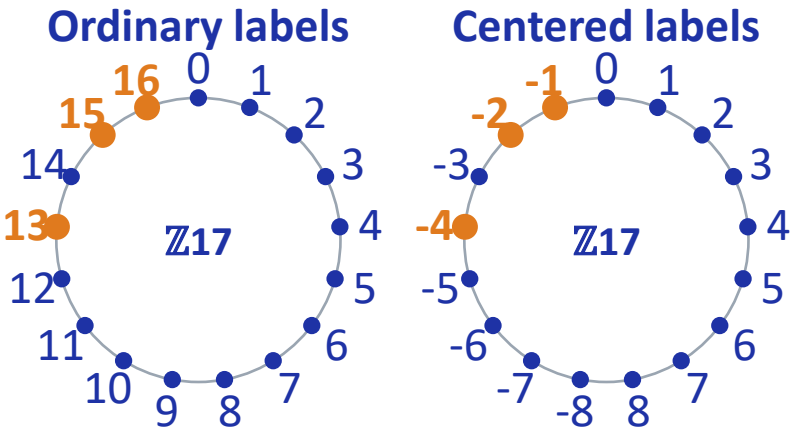
**Centered representation:**

$\mathbb{Z}_{17} = \{-8, -7, \ldots, -1, 0, 1, \ldots, 8\}$

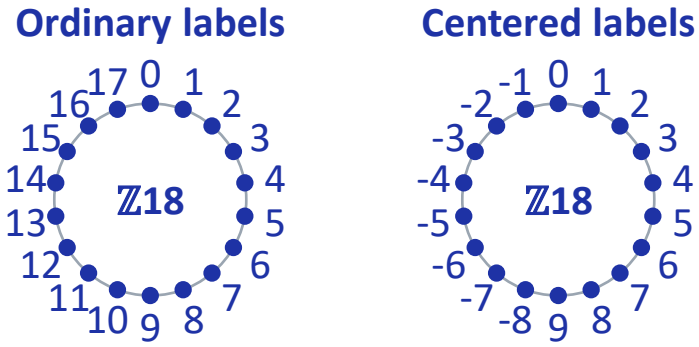
**For odd $q$:** $-(q-1)/2 \le r_c \le (q-1)/2$

**Worked examples: $\mathbb{Z}_{17}$**

$13 \equiv -4 \pmod{17}$
$16 \equiv -1 \pmod{17}$
$9 + 15 = 24 \equiv 7 \pmod{17}$
$9 - 15 = -6 \pmod{17}$
$9 \times 15 = 135 \equiv -1 \pmod{17}$




*13 ↔ −4, 15 ↔ −2, 16 ↔ −1 — centered representation changes how we write a residue, not its value modulo q.*




$\mathbb{Z}_{18} = \{-8, -7, \ldots, -1, 0, 1, \ldots, 9\}$

**For even $q$:** $-q/2 < r_c \le q/2$

# How Do We Measure the Size of a Polynomial?

***Infinity norm (maximum norm):*** *For a vector, the infinity norm is the largest absolute value of its components. For polynomials in* ***Rq****, we apply this definition to their centered coefficients.*

*If coefficients are represented modulo q, how do we decide whether a polynomial is small?*

**Step 1 — Size of a residue:**

Let $r_c$ be the centered representative of $r \in \mathbb{Z}_q$.

$$|r|_\infty = |r_c|$$

For $q = 19$: $|7|_\infty = 7$, $|18|_\infty = |-1| = 1$

**Step 2 — Size of a polynomial:**

$$|f|_\infty = \max_{0 \le i < n} |a_i|_\infty$$

Worked example — Rq = $\mathbb{Z}19[x]/(x^4+1)$, f(x) = $1+12x+3x^2+18x^3$:

$f(x) \equiv 1 - 7x + 3x^2 - x^3$

$$|f|_\infty = 7$$

**Step 3 — Size of a module vector:**

$$|a|_\infty = \max_i |a_i(x)|_\infty$$

**Coefficient breakdown:**

| coeff. | ordinary | centered | $\lvert\cdot\rvert_\infty$ |
|---|---|---|---|
| a0 | 1 | 1 | 1 |
| a1 | 12 | −7 | **7** |
| a2 | 3 | 3 | 3 |
| a3 | 18 | −1 | 1 |

*The infinity norm measures the largest centered coefficient magnitude (highlighted), not the polynomial's degree.*

# Small Polynomials — and What Happens When We Multiply Them?

A polynomial is called small when all its centered coefficients have small absolute values.

**For a positive integer η:** $S_\eta = \{f \in R_q : |f|_\infty \le \eta\}$

Example — $q = 31$: $f(x) = 1+30x+29x^2+x^4+2x^5 \equiv 1-x-2x^2+x^4+2x^5$

$|f|_\infty = 2, \quad f \in S_2$

**What happens when small polynomials are multiplied? — Rq = $\mathbb{Z}_{137}[x]/(x^4+1)$:**

$f(x) = 1+x-2x^2+2x^3 \in S_2, \quad g(x) = -2+2x^2-x^3 \in S_2$

$f(x)g(x) = -2-2x+6x^2-3x^3-5x^4+6x^5-2x^6$

$f(x)g(x) = -2-2x+6x^2-3x^3+5-6x+2x^2$

$f(x)g(x) = 3-8x+8x^2-3x^3$

$f(x)g(x) = 3+129x+8x^2+134x^3 \equiv 3-8x+8x^2-3x^3$

$|fg|_\infty = 8$

**Small f, g**
input bound: η = 2, 2

→ **Multiply in Rq**
$x^4 = -1$ reduction applied

→ **Bounded Product**
actual 8 ≤ bound 16

*Multiplying two small polynomials yields a polynomial with larger — but still bounded — coefficients: fg ∈ S8, well inside the general bound S16.*

# Exact Linear Equations Modulo q

A system of exact linear equations can be solved efficiently using modular linear algebra.

$$As = b \pmod{q}$$

A, b known; s unknown secret; q a prime modulus.

*For q = 17: 9 + 15 ≡ 7 (mod 17)  9 − 15 ≡ 11 (mod 17)*

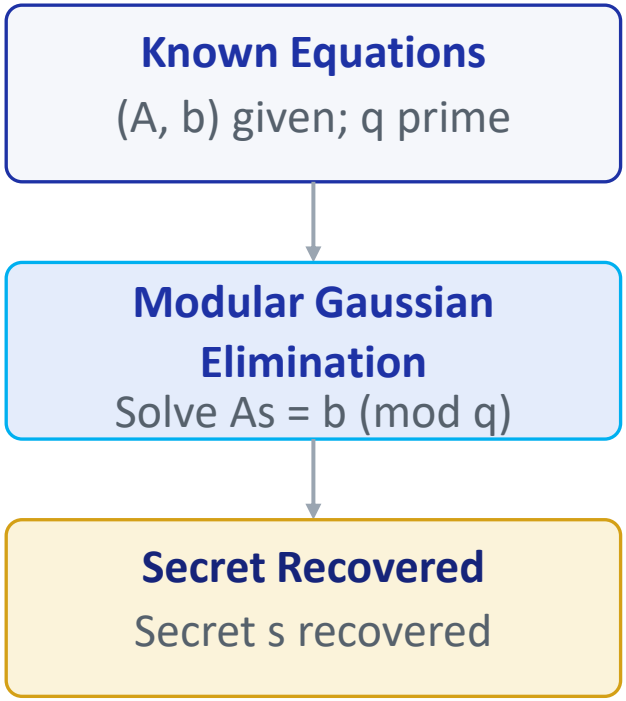
**Worked example — q = 7:**

$2s_1 + s_2 \equiv 5 \pmod 7$

$s_1 + 3s_2 \equiv 3 \pmod 7$

$(s_1, s_2) = (1, 3)$

Verification: 2(1) + 3 = 5 · 1 + 3(3) = 10 ≡ 3 (mod 7)




*When **q** is prime, modular Gaussian elimination can solve exact linear equations. Sufficient independent equations determine a unique secret; otherwise, the system may have multiple solutions or no solution.*

# Introducing Noise — Learning With Errors

What changes when a small, unknown error is added to every equation?

**Exact Equations**

$As = b \pmod q$

Known: (A - matrix, b - observations)

Unknown: s – secret vector

Method: Modular linear algebra.

**Noisy Equations**

$As + e = b \pmod q$

Known: (A - matrix, b - observations)

Unknown: (s – secret vector, e – small error vector)

Constraint: error coordinates are small.

Challenge: Recover **s** despite unknown, small errors

$$e = (e_1, e_2, \ldots, e_m)^T, \quad e_i \in [-B, B]$$

**Learning With Errors** (**LWE**): Given **A** and **b = As + e mod q**, recover the unknown secret **s**, where **e** contains small, unknown errors.

| | |
|---|---|
| $A \in \mathbb{Z}_q^{m \times n}$ | known matrix |
| $s \in \mathbb{Z}_q^{n \times 1}$ | unknown secret vector |
| $e \in \mathbb{Z}_q^{m \times 1}$ | unknown error vector |
| $b \in \mathbb{Z}_q^{m \times 1}$ | known observation vector |

$$A^{(m \times n)} \; s^{(n \times 1)} + e^{(m \times 1)} = b^{(m \times 1)} \pmod q$$

$$\begin{aligned} a_{11}s_1 + \ldots + a_{1n}s_n + e_1 &\equiv b_1 \\ a_{21}s_1 + \ldots + a_{2n}s_n + e_2 &\equiv b_2 \\ a_{31}s_1 + \ldots + a_{3n}s_n + e_3 &\equiv b_3 \pmod q \end{aligned}$$

*Ignoring the errors generally gives an incorrect system. Recovering the secret requires accounting for the unknown, constrained errors.*

# Learning With Errors — A Numerical Example

Parameters: m = 5, n = 3, q = 47, B = 2 — five equations, three secret coordinates, modulus 47, errors between −2 and +2.

$$As + e = b \pmod{47}$$

**Five equations (mod 47):**

```
27s1 + 13s2 + 13s3 + e1 ≡ 30
s1 + 46s2 + 23s3 + e2 ≡ 28
16s1 + 13s2 + 30s3 + e3 ≡ 25
18s1 + 16s2 + 19s3 + e4 ≡ 34
22s1 + 3s3 + e5 ≡ 32
```

*(all five equations taken mod 47)*

The audience can verify the first equation by hand: 27(2) + 13(15) + 13(12) + 1 = 406, and 406 ≡ 30 (mod 47).

**One verified solution:**

$$s = (2, 15, 12)^T$$

$$e = (1, 0, 2, 0, -1)^T$$

As mod 47 = $(29, 28, 23, 34, 33)^T$

(As+e) mod 47 = $(30, 28, 25, 34, 32)^T$ ✓

*Even this small example has three valid secret/error pairs satisfying the error bound — a small toy instance need not have a unique solution. One solution is shown here for clarity.*